

Solution Write-Up

Project Title

Podcast Q&A Bot using Retrieval-Augmented Generation (RAG)

Problem Statement

Podcasts are a valuable source of information, but extracting specific information from long episodes is often difficult and time-consuming. Users typically need to manually listen through lengthy recordings to find answers to particular questions.

The objective of this project was to develop an AI-powered system capable of understanding podcast content and allowing users to ask natural language questions about a podcast episode. The system should provide accurate answers along with supporting evidence and timestamps from the original podcast.

Proposed Solution

To address this problem, a Retrieval-Augmented Generation (RAG) based architecture was implemented.

The solution combines speech-to-text transcription, semantic search, vector databases, and large language models to enable intelligent question answering over podcast content.

Instead of relying solely on a language model's knowledge, the system first retrieves relevant transcript segments and then uses those segments as context for answer generation. This approach improves accuracy and reduces hallucinations.

System Architecture

Workflow:

- Download audio from YouTube podcasts
- Automatic speech-to-text transcription using Whisper
- Semantic search using vector embeddings
- FAISS vector database for fast retrieval
- AI-generated answers using Google Gemini
- Timestamp navigation to relevant podcast moments
- Supporting transcript evidence for answer verification

- Confidence score display
-

Implementation Details

1. Audio Extraction

The user provides a YouTube podcast URL.

The system downloads the podcast audio using yt-dlp and stores it locally for processing.

Technology Used:

- yt-dlp
-

2. Speech-to-Text Transcription

The downloaded audio is transcribed using OpenAI Whisper.

The transcription process generates timestamped transcript segments that preserve the temporal information of the podcast.

Technology Used:

- OpenAI Whisper

Output Example:

00:10:25 – 00:10:42

“Machine learning is a subset of artificial intelligence...”

3. Transcript Chunking

The transcript is divided into smaller segments suitable for semantic retrieval.

Each chunk contains:

- Text
- Start timestamp
- End timestamp

These chunks form the knowledge base used during question answering.

4. Embedding Generation

Each transcript chunk is converted into a dense vector representation using a Sentence Transformer model.

Embedding Model:

- all-MiniLM-L6-v2

The embeddings capture semantic meaning rather than exact keyword matches, enabling more intelligent retrieval.

Technology Used:

- Sentence Transformers
-

5. Vector Database Creation

The generated embeddings are stored inside a FAISS vector index.

FAISS enables efficient similarity search over thousands of transcript chunks.

Technology Used:

- FAISS
-

6. Semantic Retrieval

When a user asks a question:

1. The question is converted into an embedding.
2. FAISS performs similarity search.
3. Top-K relevant transcript chunks are retrieved.

This ensures that only the most relevant portions of the podcast are provided to the language model.

7. Answer Generation

The retrieved transcript chunks are provided to Google Gemini through a carefully designed prompt.

Gemini generates a concise answer using only the retrieved transcript content.

This prevents the model from relying on external knowledge and keeps responses grounded in the podcast.

Technology Used:

- Google Gemini 2.5 Flash
-

8. Answer Verification

To improve transparency and trustworthiness, the system displays:

- Generated answer
- Confidence score
- Supporting transcript chunks
- Podcast timestamp
- Direct navigation link to the relevant podcast segment

This allows users to verify the answer against the original source.

Key Features

- AI-powered podcast question answering
 - Retrieval-Augmented Generation (RAG)
 - Semantic search using embeddings
 - Timestamp-based navigation
 - Evidence-backed responses
 - Confidence score display
 - Reusable podcast indexing
 - Interactive Streamlit interface
-

Challenges Faced

Several challenges were encountered during development:

Retrieval Quality

Initially, some retrieved transcript chunks were not sufficiently relevant to user queries. This was improved by optimizing embedding generation and retrieval parameters.

Long Podcast Processing

Large podcast episodes required significant processing time for transcription and indexing. Caching mechanisms were introduced to avoid repeated processing.

Answer Grounding

Ensuring that generated answers remained faithful to the transcript was important. A retrieval-first approach was adopted to provide Gemini with relevant context before answer generation.

User Verification

To increase confidence in generated responses, transcript evidence and timestamps were included in the interface.

Results

The developed system successfully enables users to:

- Process YouTube podcasts automatically
- Ask natural language questions
- Receive context-aware answers
- View supporting transcript evidence
- Navigate directly to relevant podcast moments

The solution demonstrates the practical application of Retrieval-Augmented Generation for long-form audio content.

Technologies Used

Programming Language:

- Python

Frontend:

- Streamlit

Audio Processing:

- yt-dlp
- FFmpeg

Speech Recognition:

- OpenAI Whisper

Embeddings:

- Sentence Transformers
- all-MiniLM-L6-v2

Vector Database:

- FAISS

Large Language Model:

- Google Gemini 2.5 Flash

Environment Management:

- Python Virtual Environment
 - python-dotenv
-

Future Enhancements

Potential future improvements include:

- Multi-podcast search
 - Hybrid retrieval (keyword + semantic search)
 - Speaker identification
 - Podcast summarization
 - Conversation memory
 - Evaluation dashboard
 - Cloud deployment
-

Conclusion

This project demonstrates how Retrieval-Augmented Generation can be applied to transform long-form podcast content into an interactive and searchable knowledge source. By combining Whisper transcription, semantic retrieval using FAISS, and Gemini-based answer generation, the system provides accurate, explainable, and user-friendly access to podcast information.